

Dynamic Feedback (DFR) — Full-Stack Knowledge Transfer

Audience: Full-stack developers new to the repository
Purpose: Map components, modules, and request/data flows end-to-end
Repository: `dynamic-feedback` (Korn Ferry KfOne DFR)

1. Product context

Dynamic Feedback delivers **interactive feedback reports** in the browser (a “live report player”), with **AI-assisted chat, notes, analytics**, and optional **learning recommendations**. A **host platform** (e.g. assessment or talent systems) authenticates users, supplies assessment metadata, and typically uploads a **PDF report** to **AWS S3**. The platform converts that PDF into **page images** for smooth in-browser playback, persists state in **PostgreSQL**, and uses **RabbitMQ** plus **OpenAI** for asynchronous AI workloads. The browser receives real-time updates via **AWS IoT / MQTT**.

2. Repository layout (what lives where)

Path	Role
<code>frontend/</code>	React 18 SPA (port 3008 in architecture notes); MUI v5, Zustand, Axios wrapper, MQTT.
<code>backend/</code>	Express 4 + TypeScript API (port 8081); PostgreSQL (<code>aifedback</code>), RabbitMQ, S3, integrations.
<code>dfr_lambda/</code>	AWS Lambda: S3-triggered PDF → PNG conversion and DB <code>uploaded</code> flag (SAM/CloudFormation).
<code>db/</code>	Yuniql-style versioned SQL migrations under <code>scripts/</code> ; schema <code>aifedback</code> .
<code>.cursor/rules/</code>	Architecture and conventions (SYSTEM_ARCHITECTURE, FRONTEND, BACKEND, DATABASE, DFR_LAMBDA).

Treat **three deployable surfaces** as one system: **SPA + API + Lambda** (plus shared **DB, S3, MQ, IoT**).

3. Request routing — backend (`backend/src`)

3.1 Process entry

- `index.ts` — Initializes the **PostgreSQL pool** (`initializePostgresPool`), starts HTTP listener from `config.port` , registers **graceful shutdown** (closes server, RabbitMQ consumer/producer). Note: `registerRabbitMQConsumers()` (DFR PDF queue consumer) is **commented out** here; PDF processing may run in **Lambda** and/or other environments where that consumer is enabled.
- `app.ts` — Express application: **Helmet, CORS, Morgan**, JSON/urlencoded body limits, **CSRF-style guard** for state-changing methods without auth/API key, rate limit on `/api/dfr/user-auth` and `/api/dfr/refreshSession` , mounts `/api` → `apiRoutes` , global **notFound** and **errorHandler**.

3.2 API mount (routes/index.ts)

All HTTP APIs are under `/api` :

Prefix	Router file	Responsibility
<code>/api/dfr</code>	<code>routes/dfr.routes.ts</code>	Core DFR: auth, live report assets, notes, tracking, host integration, CMS content, reporting.
<code>/api/ai-chat</code>	<code>ai-chat/routes/index.ts</code>	AI Chat as a service: chat, samples, recommendations, summary, utilities, config, account (SSO-related).
<code>/api/iot</code>	<code>routes/iot.routes.ts</code>	IoT-related endpoints (supporting MQTT/device flows as implemented).
<code>/api/health</code>	<code>routes/health.routes.ts</code>	Health checks.

3.3 DFR routes — mental model (dfr.routes.ts)

Representative patterns:

- **User session:** GET `/user-auth`, POST `/refreshSession` → `authentication/auth.ts`.
- **Host / integration:** GET `/reporting` wrapped with `dfrIntegration`; credential and data-push endpoints use `dfrAuthenticate` / `dfrIntegration` per route block.
- **Authenticated user:** Routes like GET `/:liveReportKey/:documentKey/page/:page` use `dfrAsset` (binary-friendly, caching headers) → `main.controller` → `productionReport` service (`getLiveReportPage`).
- **Full PDF download/stream:** GET `/livereport/:documentKey/pdf` → `dfrAsset`, `getPdf`.
- **Notes / history / logging:** Pattern `/:liveReportKey/:DFRReportKey/...` with `dfrAuthenticated`.

Auth middleware (see `middleware/token.middleware.ts`, `middleware/dfr-integrations.middleware.ts`, `middleware/dfrAuth.ts`): layered model — host **X-API-KEY**, host **JWT**, end-user **DFR JWT** (and refresh), **admin**, **SAML** for specific vendors.

3.4 Controller vs service

- `controllers/main.controller.ts` — Thin mapping from Express to **service functions** (e.g. `getLiveReportPage`, `getPdf`, `notes`, `tracking`).
- `services/productionReport.ts` — Large domain module: live report **TOC**, **pages**, **S3** reads for PNG/PDF, orchestration with `commons.ts` helpers.
- `services/integrations.ts` — Host-facing flows (e.g. **presigned S3 upload URL** for the canonical PDF key under `DFR/Reports/...`).
- `services/notes.ts`, `services/commons.ts`, `services/dfr-logs.service.ts`, etc. — Feature-specific logic.

3.5 SQL and database access

- `config/database.ts` — Shared **pg** pool; `search_path` must include `aifedback`.

- `helpers/sql.helper.ts` — `SQLOne` , `SQLMany` , `SQLInsert` , `SQLUpdate` , `SQLExecute` , etc.; **always \$1, \$2 placeholders** (never `?`). Results map `snake_case` → `camelCase` and parse JSONB.

3.6 PDF pipeline (backend + optional consumer)

1. **Upload URL:** Host calls integration API; backend builds S3 key `DFR/Reports/{AppName}/{DFRUserKey}/{DFRUserAssessmentDataKey}/{ReportName}/{Language}/{ReportName}.pdf` and returns a **presigned PUT** (`services/integrations.ts`).
2. **After upload:** Either **S3** → **Lambda** (`dfr_lambda/`) **or** message to `queue.dfr.pdf` consumed by `handlers/dfrPdfHandler.ts` (if `registerRabbitMQConsumers` is enabled).
3. `dfrPdfHandler` : Validates key prefix, parses path segments, `UPDATE user_reports SET uploaded = 1` , calls `stepFunctions-convert/thumb.ts` → `extractPagesFromPDF` : download PDF from S3, **Poppler** `pdfToCairo` , upload `page-0.png` , `page-1.png` , ... (see `DFR_LAMBDA` rule for exact key layout).
4. **Serving:** Frontend requests **per-page images** via `GET .../page/:page` ; `getPdf` serves the original PDF when needed.

3.7 RabbitMQ (`rabbitmq/`)

- `producer.ts` / `consumer.ts` — Publish and subscribe; used by DFR PDF consumer and AI chat flows.
- `registerConsumer.ts` — Registers `queue.dfr.pdf` → `dfrPdfHandler` (when invoked from startup).

4. AI Chat subdomain (`backend/src/ai-chat/`)

AI Chat is intentionally **scoped** under `ai-chat/` with its own **auth**, **validation**, and **queue** wiring.

4.1 Router aggregation (`ai-chat/routes/index.ts`)

Sub-router mounted at `/api/ai-chat` :

1. `aiChatLoggingMiddleware` — Request logging for all AI chat routes.
2. **Route registration order matters:** **samples** routes are registered **before chat** routes to avoid path shadowing.
3. Modules mounted: **samples**, **chat**, **recommendations**, **summary**, **utilities**, **config**, **account** (prompts/welcome routes may be commented out in your branch).

4.2 Module map

Module path	Typical responsibility
modules/chat/	Chat CRUD, streaming workflow, <code>chat.routes.ts</code> , <code>chat.controller.ts</code> , <code>chat.service.ts</code> , <code>chatCRUD.service.ts</code> , <code>chat.workflow.ts</code> .
modules/samples/	Sample prompts/messages for UX.
modules/recommendations/	Learning recommendations (Skillsoft/KF Learn routing keys exist in queue constants).
modules/summary/	Chat summarization endpoints.
modules/utilities/	Utilities including <code>utilities.routes.ts</code> and <code>services/files.ts</code> (local static file listing/streams).
modules/config/	AI chat configuration CRUD.
modules/account/	Account/SSO-adjacent controllers (e.g. Yoodli).
modules/prompts/ , modules/welcome_messages/	Present in tree; may be partially disabled at router level.

4.3 Auth (ai-chat/auth/)

`auth.ts` , `config.middleware.ts` , `context-wrapper.ts` , `request-context.parser.ts` — Resolve **app/user context** for AI chat requests separately from generic DFR routes where required.

4.4 Queues (ai-chat/common/queue/index.ts)

- **Publishes** via wrapped `publishMessage` (logging to `rabbitmq-logger.service`).
- **Constants:** `queue.chat` , `queue.chat.summary` , `queue.learning` with routing keys for **chat.post**, **chat.summary**, **learning.kflearn**, **learning.skillsoft**.
- `initAIChatConsumers()` — Registered on startup **except when** `isLocal` . Consumers delegate to `chat.workflow` etc.

4.5 LLM integration (ai-chat/llms/gpt/openai/)

`openai.ts` , `openai.vectorstore.ts` , **types** — OpenAI API usage for chat and retrieval patterns.

5. Frontend (frontend/src)

5.1 Bootstrap

- `index.js` — React root.

- **App.js** — **BrowserRouter** ; on mount calls `useSessionStore.checkLogin('feedbackportal')` ; wraps tree with `MqttProvider` → `LiveEventsProvider` → `AppWrapperPlus` → `Routes` ; optional `OpenTelemetry` and `Pendo`.

5.2 Routing (routes/index.js)

- `useAuthRoute` — Guards authenticated routes; shows **Loading** until session resolved.
- `AuthRoute` `path="/livefeedback"` → lazy `LiveFeedback` page; unknown paths → **404** .

5.3 State — Zustand (lib/stores/)

- `useSessionStore` — Auth, theme, locale, static CMS content fetch (`/dfr/content/...`).
- `useDocumentStore` — Active live report; builds `pdfUrl` as `{apiBaseUrl}/dfr/livereport/{documentKey}/pdf` when session exists.
- `useMQTTStore` , `useAIChatStore` , `usePlayerUIStore` , `useReportStore` , `useChatSessionStore` , etc.

5.4 API layer (lib/api.js)

`apiGeneral.getDirect` / `postDirect` / ... — Central place for **base URL**, **Bearer** injection, **401** → **refresh** via `/api/dfr/refreshSession` . Avoid scattering raw `Axios` for `DFR/AI-chat` calls.

5.5 Live report UI

- **Containers:** `LiveReportsPlayerContainer.js` , `LiveReportsEditorContainer.js` — viewport lock, close handlers, embed `postMessage` for parent frame activity.
- **Player:** `LiveReportsPlayer.js` — orchestrates canvas, TOC, notes, subtitles, PDF helpers.
- **PDF-related UI:** `LiveReportPdf.js` , `LiveReportPdf.builder.js` , `lib/pdf/` , `lib/PDFDocument/` — distinguish **in-player page images** (from API) from **export/print** flows.
- **Hooks:** `useLivePages.js` , `useStructuredTOC.js` , `useLiveReportPrompt.js` , etc.

5.6 MQTT and live events

- `MqttProvider.js` , `mqtClient.js` / `lib/shared/mqtClient.js` — Connection using user context.
- `LiveEventsProvider` — Bridges MQTT into **RxJS**-style bus for live updates (e.g. chat streaming signals).

5.7 AI Chat UI

- `ui/components/AIChat/` — `AIChat.js` , samples, response buttons, dialogs.
- Hooks `useAIChatContent.js` , `useAIChatAppContext.js` , `useLiveChatContent.js` — Tie UI to `/api/ai-chat` endpoints and local store.

6. DFR Lambda (dfr_lambda/)

- **Trigger:** S3 **object created** under `DFR/Reports/*.pdf` (per Terraform / event config).
- **Handler** (`src/index.ts`): parse **strict S3 key**, update `user_reports.uploaded` , download PDF, **Poppler** to PNGs, upload `page-N.png` , cleanup temp files, close DB pool in `finally` .
- **Deploy:** SAM `template.yaml` , parameters per env under `parameters/` , CI workflow `dfr-lambda-deploy.yml` (if present in repo).

7. Database (db/)

- **RDBMS:** PostgreSQL in deployed environments; **schema aifeedback** on database `kfone` (see DATABASE rule; local docker may differ).
- **Migrations:** Version folders `v0.XX` with ordered `.sql` files; always `SET search_path TO aifeedback;` .
- **Domains (high level):** users/auth, reports and launch (`user_reports` , `user_assessment_data` , ...), report structure (pages/sections/steps + languages), analytics and plays, **AI chat** tables, prompts, **LMS/learning**, content, avatars/personas, logging.

8. Cross-cutting concerns

Concern	Where to look
Config / secrets	<code>backend/src/config/index.ts</code> , AWS Secrets Manager
Logging	<code>kfone-common-library</code> <code>Logger</code> / <code>logger/logger.service.ts</code> ; AI chat <code>ai-chat/common/utils/logger.ts</code>
Validation	<code>Joi</code> + <code>express-joi-validation</code> on DFR routes; per-module validation in <code>ai-chat</code>
SSO / SAML	<code>helpers/sso-sp.helper.ts</code> , <code>managers/sso-*.ts</code> , <code>constants/ssoConstants.ts</code> , account modules
Scores / assessments	<code>scores/</code> , <code>services/productionReport.ts</code> , <code>parsePdfLinks.ts</code> , <code>parsePDF.ts</code>

9. Environments and delivery (summary)

- **Lifecycle:** `sbx` → `dev` → `dv2` → `qa1` → `stg` → `prd` → `dem` (per architecture rule).
- **Regions:** `us-east-1`, `eu-central-1`, `eu-west-1`, `us-west-2`, `ap-southeast-2`.
- **CI/CD:** GitHub Actions, often via `HayGroup/kfone-common` reusable workflows; container registry **JFrog** (`kfydigital.jfrog.io`).